

Free space Management

1. Defragmentation / Compaction (Simple English)

What is the problem?

In **dynamic partitioning**, memory gets broken into many small free spaces. Even if total free memory is enough, a big process cannot fit because the free space is scattered. This is called **external fragmentation**.

What is compaction (defragmentation)?

Compaction means **moving all running processes close together** so that **all free memory comes to one single block**. After this, the scattered free spaces are merged into one big free space.

Why do we do it?

After compaction, **bigger processes can be stored easily** because we now have one large free memory block instead of many small ones.

Drawback:

The system becomes **slow for some time** because moving processes in memory takes time and effort.

Real-World Example

Imagine a **train compartment with passengers** sitting randomly:

- Some seats are empty here and there.
- You want to seat a **family of 6 together**, but there is no continuous block of 6 empty seats.
- So, the conductor asks some passengers to **shift seats** and sit close.
- Now all empty seats come together, and the family can sit.

This **shifting of passengers** is like **compaction**, and the **group of empty seats** is like **merged free memory**.

In short

Problem: Free memory is scattered (external fragmentation)

Solution: Move processes → merge free space (compaction)

Cost: System efficiency reduces temporarily

This is why compaction is also called **defragmentation**.

2. How free space is stored / represented in OS

When programs are loaded and removed from memory, **empty spaces (holes)** are created.

The Operating System keeps track of these empty spaces using a **Free List**.

Free List

- It is stored as a **Linked List**
- Each node stores:
 - Starting address of free memory
 - Size of free memory (hole)

So basically, OS knows **where free memory is and how big it is** by using this linked list.

Real-world example

Think of a **hotel**:

- Each empty room is a free hole
- The hotel manager keeps a **list of empty rooms** with room numbers and sizes
This list is just like the **free list** in OS.

3. How OS satisfies a request of size n from free holes?

When a new process comes and asks for memory of size n,
the OS looks into the **free list** and uses **allocation algorithms**.

a. Memory Allocation Algorithms

1. First Fit

Rule:

Allocate the **first hole** that is **big enough**.

Points:

- Very **simple**
- **Fast**
- Less searching → less time

Problem:

- Leaves small holes at the beginning

Real-world example

In a classroom, the teacher seats a student on the **first empty bench** that fits.
No checking of all benches → just sit quickly.

2.Next Fit

Rule:

Same as First Fit

But search **starts from the last allocated hole**, not from the beginning.

Points:

- Improvement over First Fit
- Fast
- Reduces overload on starting memory

Real-world example

Hotel manager starts giving rooms from **where the last guest was placed**, not from room no. 1 every time.

3.Best Fit

Rule:

Allocate the **smallest hole** that can fit the process.

Points:

- Reduces **internal fragmentation**
- But creates many **tiny free holes**
- **Slow** (checks full list)

Real-world example

You choose the **smallest possible box** to pack a gift.
But later, many useless small boxes are left.

4.Worst Fit

Rule:

Allocate the **largest hole** available.

Points:

- Leaves big free spaces
- Slow (full list search)
- Big holes can serve future big processes

Real-world example

Put a small bag in the **largest suitcase**, so medium bags can still fit later.

Table

Algorithm	Idea	Speed	Fragmentation
First Fit	First suitable hole	Fast	External
Next Fit	Continue from last	Fast	External
Best Fit	Smallest suitable hole	Slow	External (many small holes)
Worst Fit	Largest hole	Slow	Less external

Points to remember

- **Free space** → Stored using **Free List (Linked List)**
- **Memory allocation** → Done using **First Fit, Next Fit, Best Fit, Worst Fit**